1 Introduction

1.1 Motivation

In modern vision-language systems, self-checking is becoming more important, particularly in tasks such as image captioning, visual question answering, and robotic perception. Although vision-language models are capable of generating fluent captions, they can still generate hallucinations or incorrect object descriptions.

It is still difficult for ai systems to detect these mistakes, as many sentence embedding models focus more on overall sentence meaning than exact object details. We discuss whether semantic similarity can serve as a simple method for detecting errors in generated subtitles.

1.2 Problem Statement

We investigated whether errors in image captions generated by vision-language models could be detected using sentence similarity checks.

- Can semantic similarity detect incorrect captions?
- How much do small word changes affect Sentence-BERT similarity?
- Can this method work as a simple fault detection system?

2 Related Work

2.1 Vision-Language Models

Vision-language models have improved quickly in recent years. BLIP introduced a framework for vision-language understanding and generation, it performs exceptionally well on image captioning and VQA tasks. CLIP learned visual concepts from natural language supervision, enabling zero-shot transfer to many tasks. LLaVA demonstrated instruction-following capabilities for vision-language tasks.

Even though VLMs work well in many tasks but they still have some weaknesses. They may describe objects that are not actually in the image and may give different results after small changes to the input, or make mistakes on simple reasoning tasks. These problems can become more serious in multi-agent systems because one error may affect other agents as well.

3 Methodology

3.1 Overview

We used three functional components: a generator, a perturbation module, and a critic.

3.2 Framework Components

3.2.1 Generator

The generator creates a caption for the input image and we use deterministic generation so the output stays consistent.

Given an input image I then the generator produces:

$$c_g = G(I)$$

where G is the generator model and c_g is the generated caption.

3.2.2 Perturbation Module

We introduce errors into the generated image descriptions by modifying specific words within them. After the generator outputs a description, the framework replaces randomly selected words within it with incongruous terms—such as "dog," "car," "tree," or "building."

After generating the caption, the system modifies some words to create an incorrect version. The verifier then produces a modified caption

$$c_v = P(c_g)$$

where P represents the perturbation function.

This approach helps us test whether similarity scores can detect caption errors. Although this is not a fully independent multi-agent system, but it operates faster and requires fewer computational resources.

3.2.3 Critic

The critic compares the two captions and checks if they are similar enough in meaning or not. when the similarity score is too low, the system marks it as a possible fault. The critic computes:

$$s = \text{Sim}(c_g, c_v)$$

where Sim is a semantic similarity function. The fault detection decision is:

$$\text{fault} = \begin{cases} \text{True} & \text{if } s < \tau \\ \text{False} & \text{otherwise} \end{cases}$$

where τ is a predefined similarity threshold. We evaluated multiple threshold settings (0.7, 0.8, and 0.9) to analyze the sensitivity of the framework.

3.3 Semantic Similarity Computation

The critic compares the two captions and checks whether they are similar enough in meaning. If the similarity score is too low, the system marks it as a possible fault. The critic computes:

$$\text{Sim}(c_g, c_v) = \frac{\mathbf{e}_g \cdot \mathbf{e}_v}{\|\mathbf{e}_g\| \|\mathbf{e}_v\|}$$

where $\mathbf{e}_g = \text{SBERT}(c_g)$ and $\mathbf{e}_v = \text{SBERT}(c_v)$ are the embeddings of the two captions. This is the cosine similarity, ranging from -1 to 1, though in practice for natural language it ranges from 0 to 1.

Algorithm 1 Fault Detection Algorithm

Require: Input image I, threshold τ
Ensure: Fault flag f, similarity score s
  $c_g \leftarrow \text{Generator}(I)$
  $c_v \leftarrow \text{InjectFault}(c_g)$
  $\mathbf{e}_g \leftarrow \text{SBERT}(c_g)$
  $\mathbf{e}_v \leftarrow \text{SBERT}(c_v)$
  $s \leftarrow \text{cosine\_similarity}(\mathbf{e}_g, \mathbf{e}_v)$
  if $s < \tau$ **then**
    $f \leftarrow \text{True}$
  else
    $f \leftarrow \text{False}$
  end if
  return (f, s)

4 Experimental Setup

4.1 Dataset

We conduct experiments using a small collection of publicly accessible images obtained from Unsplash. The dataset includes 20 sample images collected from Unsplash including animals,indoor scenes, outdoor environments, transportation, and human activities.. These images contain different types of scenes, such as animals, indoor and outdoor places, vehicles, and people doing activities.To create more test samples for evaluation, we generated several synthetic perturbations for each image-caption pair. This setup allowed us to run experiments more quickly without needing a very large dataset.

4.1.1 BLIP-base

BLIP-base [1] is a 140M parameter model for image captioning. It uses a vision transformer (ViT) as the image encoder and a BERT-based text decoder. We use the version fine-tuned on COCO captioning, which achieves a CIDEr score of approximately 113 on the COCO test set.

4.1.2 Sentence-BERT (all-MiniLM-L6-v2)

This model is used to compare sentence similarity. It converts text into embeddings and is fast enough to run on Google Colab.

4.2 Evaluation Metrics

We report the following metrics:

- Mean similarity: Average cosine similarity across all samples
- Standard deviation: Spread of similarity scores
- Minimum similarity: Lowest similarity score observed
- Maximum similarity: Highest similarity score observed
- Fault detection rate: Percentage of samples with similarity below the threshold of 0.7

4.3 Hardware

Experiments were run on Google Colab.

5 Results

5.1 Quantitative Results

For comparison, we also included a simple baseline method that compares each generated caption with a fixed unrelated caption . This baseline provides a lower-bound similarity reference for evaluating the framework.

Table 2 presents the overall results of our experiment on 100 image-caption pairs.

Table 3 presents the effect of different threshold values on fault detection sensitivity.

Threshold	Detection Rate
0.7	5%
0.8	25%
0.9	65%

Table 1: Threshold sensitivity analysis

Table 2: Overall fault detection results

Metric	**Value**
Total samples	100
Detected faults	2
Undetected faults	98
Fault detection rate	5%

Table 3: Similarity statistics

Metric	**Value**
Mean similarity	0.856
Standard deviation	0.080
Minimum similarity	0.643
Maximum similarity	1.000

The results showed that most modified captions were still very similar to the original captions even after the word changes. Only one sample fell below the threshold of 0.7, resulting in a relatively low fault detection rate.

5.2 Visualizations

Figure 1 shows the distribution of semantic similarity scores produced by the framework after synthetic fault injection.

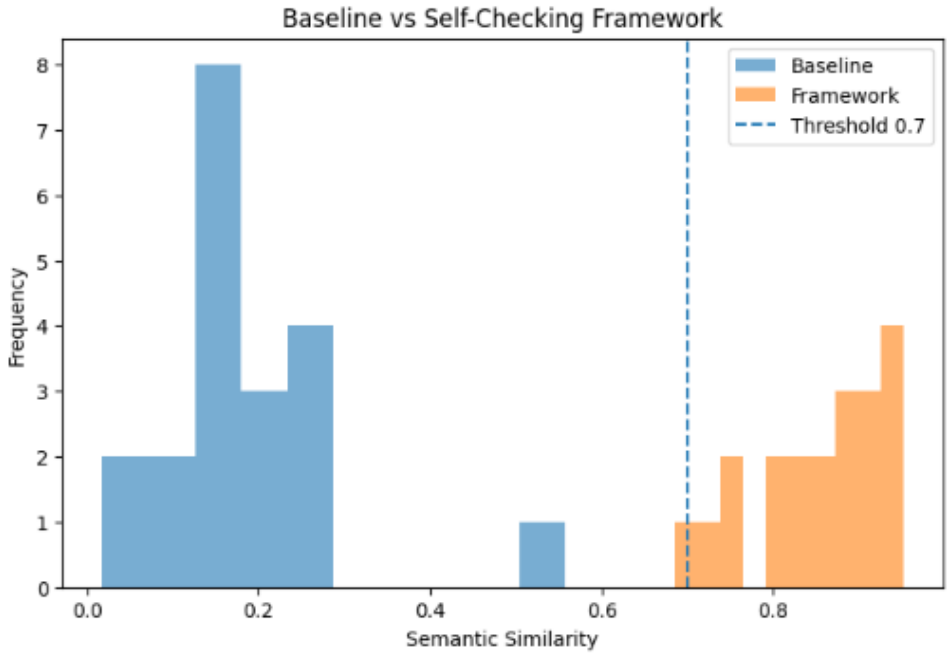

Figure 1: Distribution of semantic similarity scores with threshold at 0.7.

Most similarity scores stayed high, usually between 0.75 and 1.0. Even after changing some words, the captions still looked similar to the original ones.

Only a few samples were below 0.7. This means Sentence-BERT still gives high similarity scores when only small words are changed.

These results show that changing only a few words is not enough to make the captions very different.

6 Discussion

6.1 Interpretation of Results

The experimental results showed that Sentence-BERT was not very sensitive to small word changes. Replacing individual words such as ‘cat” with ‘dog” or ‘table” with ‘car” often still produced similarity scores above 0.8.

The model seems to care more about the general sentence meaning than exact object details. So the system may fail to detect small caption mistakes or incorrect object.

Although the framework successfully detected a small number of strongly inconsistent captions, most injected faults remained undetected under the current threshold setting.

6.2 Effect of Threshold

The threshold value will had a noticeable effect on fault detection performance. When the threshold was set to 0.7, the framework detected only 5% of the injected caption faults. After increasing the threshold to 0.8, the detection rate increased to 25%. With a threshold of 0.9, the detection rate further increased to 65%. This shows that higher thresholds make the system more sensitive to caption changes and perturbations. However, using a very high threshold may also increase the number of false positives, since some captions that are still semantically reasonable could be

classified as faults. Overall, the experiments showed that the threshold value had a strong effect on fault detection performance.

6.3 Challenges

Two main challenges emerged from our experiment.

The vision-language models remain computationally expensive, limiting the scale and diversity of our experiments. Although the current study uses 100 image-caption pairs, the underlying image set is relatively small and repeated across samples. Running the same experiment on 500+ images would require significantly more resources.

Defining what counts as a "fault" in generated outputs is inherently subjective. Two captions may describe the same scene while emphasizing different objects or attributes. Whether such differences should be considered faults depends on the application context. A high-precision system may require exact object consistency, while a more flexible system may tolerate minor semantic variation.

6.4 Limitations

Our current implementation has several limitations:

- The verifier does not use an independent vision-language model and instead relies on synthetic fault injection
- The evaluation uses a small repeated image set rather than a large benchmark dataset
- Sentence-BERT alone could not reliably detect small object changes in captions
- The similarity threshold was manually selected rather than optimized using validation data
- The framework was evaluated only on image captioning tasks
- The framework was not compared against other semantic verification or fault detection methods.

7 Conclusion and Future Work

7.1 Feasibility

The project code is available on GitHub. It includes the caption generation code, similarity calculation, and experiment scripts used in this project.

GitHub repository: https://github.com/lxf6323210-hash/59866-final-project

7.2 Conclusion

This paper we investigate a self-inspection framework for detecting semantic errors in image captioning systems. Using captions generated by BLIP, synthetic perturbations, and Sentence-BERT similarity analysis, we examined whether semantic similarity could identify injected inconsistencies in the descriptions.

Experimental results showed that most injected faults were not detected because Sentence-BERT embeddings remained highly similar despite local word substitutions. The results showed that semantic similarity by itself was not enough to reliably detect caption errors.

7.3 Future Work

Several directions remain for future investigation:

- **Different architectures**: Use different base models for generator and verifier (e.g., BLIP for generator, LLaVA for verifier)
- **Task diversity**: Test on visual question answering (VQA) and image-text retrieval
- **Scale**: Evaluate on 500+ images to obtain statistically significant results
- **Threshold tuning**: Optimize τ using a validation set with ground truth labels
- **Recovery mechanisms**: Implement fallback strategies when faults are detected (e.g., query a third agent, request human input)

References

[1] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation," in *ICML*, 2022.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *EMNLP*, 2019.

[3] T. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.

[4] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," in *NeurIPS*, 2023.

[5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," in *ICML*, 2021.

[6] Y. Shoham and K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

[7] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh, "VQA: Visual Question Answering," in *ICCV*, 2015.

[8] T. G. Dietterich, "Ensemble Methods in Machine Learning," in *Multiple Classifier Systems*, 2000.

8 Project Timeline and Task Allocation

8.1 Timeline

Week 1: Literature review on multi-agent systems and vision-language models; formulation of the research question.

Week 2: Design of the self-checking framework and semantic consistency mechanism (generator, verifier, critic) and selection of models (BLIP and Sentence-BERT).

Week 3: Implementation of the system, including image caption generation and semantic similarity computation.

Week 4: Execution of experiments on sample images and initial analysis of results.

Week 5: Visualization of results and evaluation of system performance; dataset feasibility validation.

Week 6: Refinement of methodology, report writing, and final revision.